

Algoritm: Divizorii unui numar

1. Definitie si rol

Un divizor al unui numar intreg n ($n \neq 0$) este un numar intreg d astfel incat $n \% d == 0$. La bac, cel mai des se cer divizorii pozitivi ai unui numar pozitiv.

2. Ideea algoritmica

Pentru a gasi toti divizorii pozitivi ai lui n , se pot testa toate valorile d de la 1 la n . O varianta eficienta testeaza doar pana la $\sqrt{n}$, folosind faptul ca daca d divide n , atunci si n/d divide n .

3. Implementare C++ (varianta simpla: 1..n)

```
#include <iostream>
using namespace std;

int main(){
    int n;
    cin >> n;
    if(n == 0) return 0;
    if(n < 0) n = -n;

    for(int d = 1; d <= n; d++){
        if(n % d == 0) cout << d << " ";
    }
    return 0;
}
```

4. Implementare C++ (varianta eficienta: 1.. $\sqrt{n}$), fara STL

Aceasta varianta afiseaza divizorii in perechi (d si n/d). Ordinea de afisare poate fi neordonata daca se afiseaza imediat.

```
#include <iostream>
using namespace std;

int main(){
    int n;
    cin >> n;
    if(n == 0) return 0;
    if(n < 0) n = -n;

    for(int d = 1; 1LL*d*d <= n; d++){
        if(n % d == 0){
            cout << d << " ";
            if(d != n / d) cout << (n / d) << " ";
        }
    }
    return 0;
}
```

5. Exemple (fara rezolvari)

- $n = 12$ -> divizori: 1 2 3 4 6 12 - $n = 13$ -> divizori: 1 13

6. Complexitate

Varianta 1..n: $O(n)$. Varianta 1.. $\sqrt{n}$: $O(\sqrt{n})$. Memorie: $O(1)$.

7. Observatii importante

Pentru n negativ se foloseste valoarea absoluta. Pentru $n = 0$, notiunea de divizor nu este tratata in mod uzual la bac.

8. Greseli frecvente

- uitarea cazului n negativ;
- folosirea limitei $d*d \leq n$ fara tip mai mare (overflow la $d*d$);
- dublarea divizorului cand $d == n/d$.

9. Checklist de BAC

- stii definitia divizorului;
- stii testarea cu %;
- stii varianta pana la $\sqrt{n}$ si perechile $(d, n/d)$.